

Challenge:	The Goblin's Hidden Path
Category:	Web Exploitation (Local File Inclusion / Path Traversal) & Cryptography
Difficulty:	Moderate
Tools Used:	Browser, CyberChef (or Python/Terminal)

Summary:

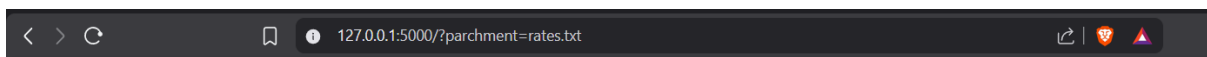
This challenge demonstrates a Local File Inclusion (LFI) vulnerability in a document retrieval system. By manipulating the parchment parameter, we can traverse the directory structure to uncover hidden files. After encountering several decoys, a specific memo reveals the path to a restricted log and the cryptographic pipeline used to secure it. Retrieving the log provides an encrypted string that must be decrypted using a sequence of ROT13, Vigenère (key "MAGIC"), and Base64 decoding to reveal the final flag.

Problem Statement:

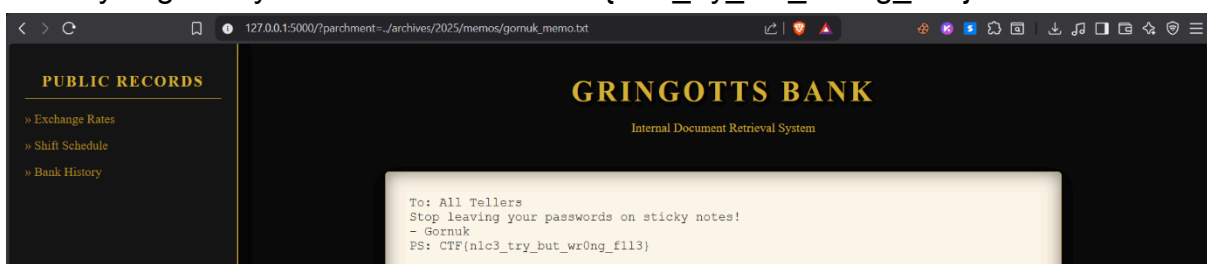
During a scheduled audit of Gringotts' internal document retrieval system, irregular responses were observed from the exchange-rate portal designed by Gornuk. The system appears to process file requests exactly as provided—without questioning where those paths may lead. If an auditor were to guide the locator beyond its intended boundaries, they might uncover restricted internal records buried deeper within the vault structure. However, Gornuk anticipated intrusion. Sensitive entries have been encrypted in layered transformations—concealed behind multiple reversible operations. Trace the path. Retrieve the record. Then unravel what was deliberately obscured.

Steps to Solve:

1. Open the challenge website interface for the Gringotts Bank Internal Document Retrieval System.
2. Observe the URL parameter used to fetch public records, which looks something like `/?parchment=rates.txt`.



3. Recognize that the application might be vulnerable to Local File Inclusion (LFI) or Path Traversal if it blindly fetches files without sanitizing `../` characters.
4. Begin mapping the internal directory by guessing standard folder structures and attempting to traverse backwards.
5. Navigate to `/?parchment=../archives/2025/memos/gornuk_memo.txt`. The response reveals a decoy flag left by Gornuk for tellers: `CTF{n1c3_try_but_wr0ng_f113}`.



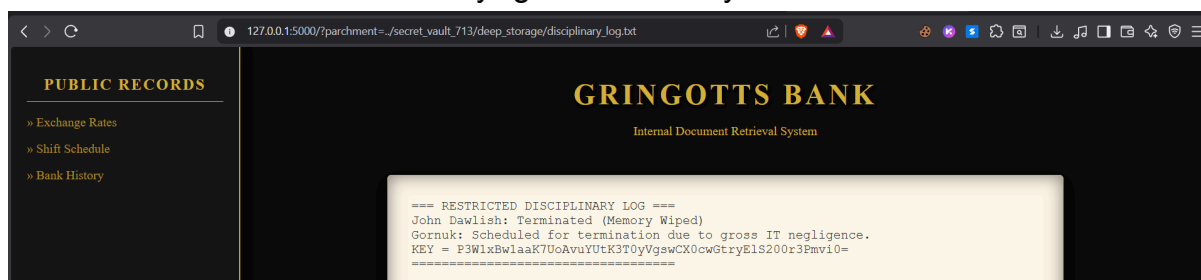
6. Continue exploring the file structure. Navigate to `/?parchment=../archives/2024/logs/error.log`. This reveals another trap containing a fake flag: `CTF{y0u_th0ught_1t_w4s_h3r3_d1dnt_y0u}`.



7. Keep probing the archives and navigate to `/?parchment=../archives/memos/gornuk_memo.txt`.

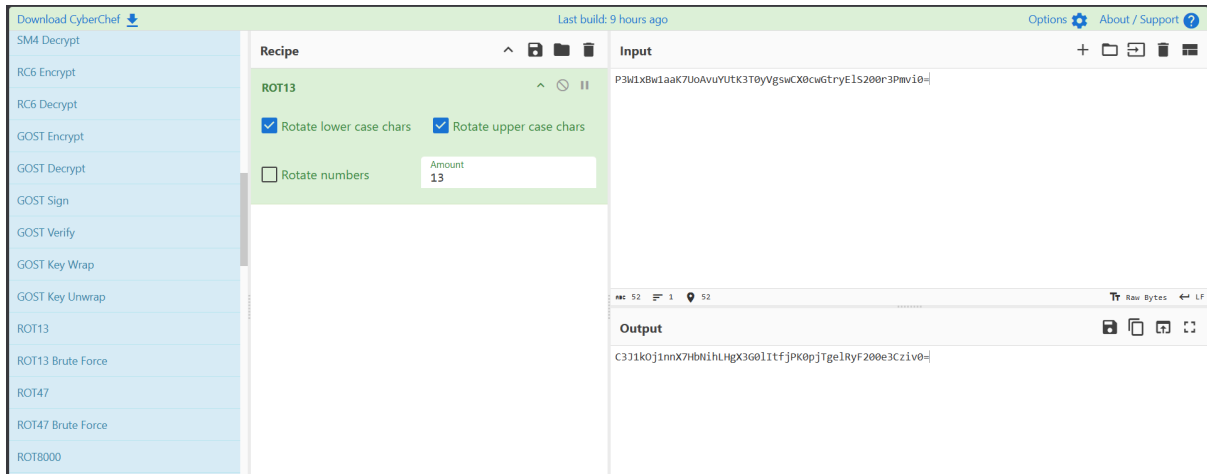


8. This file contains a critical message from Gornuk, Head Systems Architect. It reveals two vital pieces of intelligence:
- The exact internal path to the highly restricted disciplinary log: `/secret_vault_713/deep_storage/disciplinary_log.txt`.
 - The cryptographic layers used to protect the log: It is wrapped in a standard 64-character format (Base64), locked with a repeating keyword ("MAGIC") using a Vigenère cipher, and shifted by exactly half the alphabet (ROT13).
9. Use the newly discovered path to craft the final LFI payload in the URL bar: `/?parchment=../secret_vault_713/deep_storage/disciplinary_log.txt`.
10. The server successfully fetches the restricted log, which contains the encrypted string: `KEY = P3WlxBwlaak7UoAvuYUtK3T0yVgsWCX0cwGtryEIS200r3Pmvi0=`

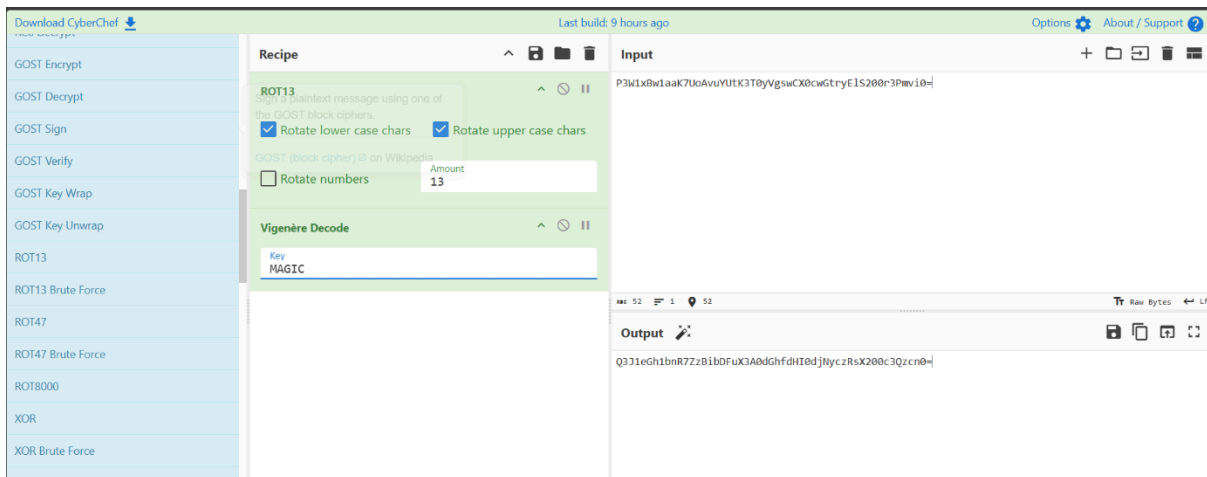


11. Now, reverse the cryptographic pipeline described in Gornuk's memo. Since the encryption was applied in the order of Base64 → Vigenère → ROT13, decryption must be executed in the exact reverse order.
12. Copy the encrypted string: `P3WlxBwlaak7UoAvuYUtK3T0yVgsWCX0cwGtryEIS200r3Pmvi0=`
13. Open CyberChef (or utilize a custom Python script).

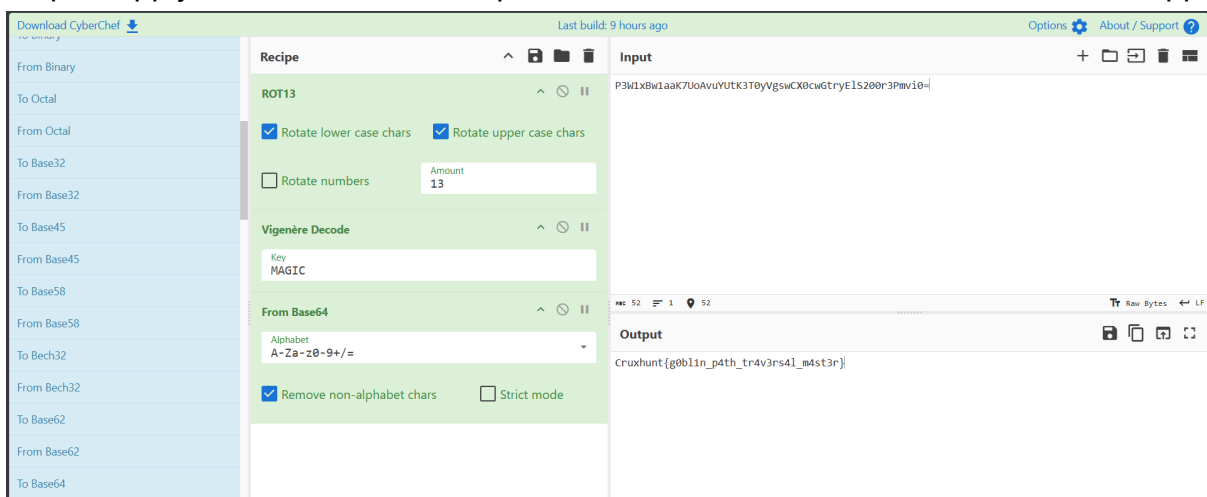
14. Step 1: Apply the "ROT13" decoding operation to reverse the "shifted by half the alphabet" layer.



15. Step 2: Apply the "Vigenère Decode" operation using the key MAGIC to unlock the repeating keyword cipher.



16. Step 3: Apply the "From Base64" operation to reverse the 64-character format wrapping.



17. The resulting plaintext reveals the final flag.

Final Flag:

Cruxhunt{g0bl1n_p4th_tr4v3rs4l_m4st3r}